

VERSION 1.0

JANUARY 2026



DATA MINING

MODULE 5 - REGRESSION & CLASSIFICATION

Authored by:

Yufis Azhar, Ir., S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY of MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
PRACTICAL OBJECTIVES.....	3
PREPARATION.....	3
KEYWORDS.....	3
1. Introduction to Supervised Learning.....	4
1.1 The Mapping Function.....	4
1.2 Taxonomy of Supervised Learning.....	4
2. Data Splitting Strategies.....	5
2.1 Training vs Testing Sets.....	5
2.2 Technical Implementation.....	6
3. Regression: Predicting Continuous Trends.....	7
3.1 Linear Regression Mechanics.....	7
3.2 Evaluation Metrics.....	8
How to Interpret the Regression Output Above:.....	11
4. Classification: Logistic Regression and Performance Metrics.....	11
4.1 Logistic Regression.....	11
4.2 The Confusion Matrix.....	13
CODELAB (20%).....	16
LAB ASSIGNMENT (80%).....	17
GRADING CRITERIA & DETAILS.....	18



INTRODUCTION

PRACTICAL OBJECTIVES

1. Differentiate between Regression and Classification tasks based on the nature of the target variable (y)
2. Implement data splitting using Scikit-Learn to create independent training and testing sets for model validation
3. Build a Simple Linear Regression model to predict continuous numerical trends
4. Construct a Logistic Regression model for binary classification problems
5. Evaluate model performance using domain-standard metrics such as R^2 and RMSE for regression, and Accuracy, Precision, and Recall for classification

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
 رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Hardware:
 - Laptop/Computer
3. Software (Choose one):
 - Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook
4. Tutorial
 - Using Google Colab:
https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
 - Using Jupyter in Visual Studio Code:
<https://youtu.be/HOq13z1w11A?si=RhEY2QY5bir-dzEH>
 - Using Jupyter Notebook:
https://youtu.be/HQaCF2F_MRo?si=k6ubuGuCg-VS0tQe
 - Install Anaconda:
https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt
5. Source Code (Google Colaboratory):
 - <https://colab.research.google.com/drive/1KYhBJXlevLELqhP308j5OCbLXP4gRCpl?usp=sharing>
6. Access Dataset:
 - https://drive.google.com/drive/folders/1Bo9iB_9YgeSdW6j8mJ5EC0BWZDf4REIW?usp=sharing
7. Strong motivation and commitment to learning 😊

KEYWORDS

Regression, Classification, R^2 , RMSE, Accuracy, Precision, Recall



1. Introduction to Supervised Learning

Supervised Learning is the most common branch of Machine Learning. It is based on the concept of learning by example. In this paradigm, we provide an algorithm with a dataset that includes both the inputs and the correct desired outputs (labels). By "supervising" the model with known answers, we allow it to learn the underlying patterns so it can make accurate predictions on new, unseen data.

1.1 The Mapping Function

The fundamental goal of a Supervised Learning algorithm is to find a Mapping Function (f) that connects the input variables (X) to the target output variable (y).

- **Input Variables (X)**: Also known as Features, Independent Variables, or Attributes. These are the data points we use to make a prediction (e.g., square footage of a house).
- **Output Variable (y)**: Also known as the Target, Label, or Dependent Variable. This is what we want to predict (e.g., the price of the house).

The mathematical representation is:

$$y = f(X) + \epsilon$$

Note: The symbol ϵ (epsilon) represents random error or "noise" that is inherent in any real-world data and cannot be fully captured by the model. The objective of the learning process is to minimize the difference between the model's prediction and the actual label.

1.2 Taxonomy of Supervised Learning

Supervised Learning is broadly categorized into two main types based on the nature of the target variable (y). Choosing the right category is the first and most important step in building a model.

A. Regression (Continuous Output)

In Regression, the model predicts a quantity or a continuous numerical value. If you can ask "How much?" or "How many?", it is likely a regression problem.

Characteristic: The output has an infinite number of possible values within a range.



Examples:

1. Predicting the price of a used car based on its mileage and year.
2. Predicting the temperature for tomorrow.
3. Estimating the revenue of a company for the next quarter.

B. Classification (Categorical Output)

In Classification, the model predicts a label or a discrete category. The goal is to assign the input data into specific "classes."

Characteristic: The output belongs to a predefined, finite set of groups.

Sub-types:

- Binary Classification: Only two possible classes (e.g., Yes/No, Spam/Not Spam, Fraud/Legit).
- Multi-class Classification: More than two classes (e.g., Classifying an image as a Dog, Cat, or Bird).

Examples:

- Determining if a bank transaction is Fraudulent or Legitimate.
- Classifying an email as Spam or Inbox.
- Predicting whether a patient has a specific Disease (Positive/Negative).

2. Data Splitting Strategies

In the world of Supervised Learning, the ultimate goal is not to have a model that performs perfectly on data it has already seen, but a model that can accurately predict values for unseen data. To measure this, we must split our data.

2.1 Training vs Testing Sets

The core philosophy of Supervised Learning is to create a model that can Generalize. Generalization is the model's ability to make accurate predictions on new, unseen data. To measure this, we partition our dataset into two primary subsets:

- **Training Set:** This is the portion of data (typically 70% to 80%) that the algorithm "studies." The model uses this data to find patterns and adjust its mathematical parameters to minimize error.
- **Testing Set:** This is the "final exam" (typically 20% to 30%). It is kept strictly separate from the model during the training phase. Once the model is built, we use the test set to evaluate its actual performance.



Why is this necessary?

If we test the model on the training data, it might appear to have 100% accuracy simply because it "memorized" the answers. This failure is known as Overfitting. A good model should learn the underlying "signal" of the data, not the random "noise."

2.2 Technical Implementation

To implement data splitting effectively in Python, we use the `train_test_split` function from the `sklearn.model_selection` library. This ensures the data is shuffled and divided correctly.

Key Parameters:

1. `test_size`: Determines the percentage of data allocated for testing (e.g., 0.2 for 20%).
2. `random_state`: Because computers pick rows randomly, using a fixed number (like 42) ensures that every time you run the code, you get the exact same split. This is crucial for reproducibility in research.
3. `stratify`: Used in classification tasks. It ensures that the ratio of classes (e.g., "Sick" vs. "Healthy") in the training and testing sets is identical to the original dataset, preventing a biased split.

Example:

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

wine = load_wine()

df_wine = pd.DataFrame(wine.data, columns=wine.feature_names)
df_wine['target'] = wine.target

df_binary = df_wine[df_wine['target'].isin([0, 1])]

X_class = df_binary.drop('target', axis=1)
y_class = df_binary['target']

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_class,
    y_class,
```



```
test_size=0.2,
random_state=42,
stratify=y_class
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_c)
X_test_scaled = scaler.transform(X_test_c)

print("Wine Binary Dataset Successfully Loaded and Split!")
print(f"Training Data Shape : {X_train_scaled.shape}")
print(f"Testing Data Shape : {X_test_scaled.shape}")
```

Output:

```
... Wine Binary Dataset Successfully Loaded and Split!
   Training Data Shape : (104, 13)
   Testing Data Shape  : (26, 13)
```

3. Regression: Predicting Continuous Trends

Regression analysis is a statistical process for estimating the relationships between a dependent variable (target) and one or more independent variables (features). In Data Mining, we use it to predict future trends, such as stock prices, weather temperatures, or sales growth.

3.1 Linear Regression Mechanics

Linear Regression is the simplest and most widely used regression algorithm. It assumes that the relationship between the input X and the output y can be modeled with a straight line.

A. The Linear Equation

The model tries to find the best values for the coefficients (weights) to fit the data points. The formula for a simple linear regression is:

The mathematical representation is:



$$y = mx + c$$

- y : The predicted value (Target).
- x : The input value (Feature).
- m : The Slope (Gradient), representing how much y changes for every unit change in x .
- c : The Intercept, representing the value of y when x is zero.

B. Ordinary Least Squares (OLS)

To find the "Best Fit Line," the algorithm uses a method called Ordinary Least Squares. It calculates the distance between each actual data point and the predicted line (this distance is called the Residual). The "best" line is the one that minimizes the sum of all these squared residuals.

3.2 Evaluation Metrics

Once the model is trained, we need to measure how "wrong" its predictions are. Unlike classification (where we use accuracy), in regression, we measure the distance between the predicted value and the actual value.

1. Mean Squared Error (MSE)

MSE calculates the average of the squares of the errors. Squaring the error is important because it punishes larger errors more heavily and ensures all values are positive.

- Formula: Average of $(\text{Actual} - \text{Predicted})^2$
- Interpretation: The closer the MSE is to 0, the better the model.

2. R-Squared (R^2 Score)

The R^2 score, also known as the Coefficient of Determination, tells us how much of the variation in the target variable is explained by the model.

- Range: 0 to 1 (or 0% to 100%).
- Interpretation: An R^2 of 0.80 means that 80% of the variance in the target data is explained by your model. A score of 1.0 is a perfect fit.

Example:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```




```
url_reg =
"https://raw.githubusercontent.com/mwaskom/seaborn-data/master
/tips.csv"
df_tips = pd.read_csv(url_reg)

X_reg = df_tips[['total_bill', 'size']]
y_reg = df_tips['tip']

X_train_r, X_test_r, y_train_r, y_test_r =
train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

model_reg = LinearRegression()
model_reg.fit(X_train_r, y_train_r)

y_pred_r = model_reg.predict(X_test_r)

print(f"Mean Squared Error: {mean_squared_error(y_test_r,
y_pred_r):.2f}")
print(f"R-Squared Score: {r2_score(y_test_r, y_pred_r):.2f}")
```

Output:

```
Mean Squared Error: 0.65
R-Squared Score: 0.48
```

Example:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(7, 4.5))

plt.scatter(
    X_test_r['total_bill'],
    y_test_r,
    color='blue',
    alpha=0.6,
    label='Actual Data'
)

sort_axis = X_test_r['total_bill'].argsort()
```



```
plt.plot(
    X_test_r['total_bill'].iloc[sort_axis],
    y_pred_r[sort_axis],
    color='red',
    linewidth=2,
    label='Best Fit Line'
)

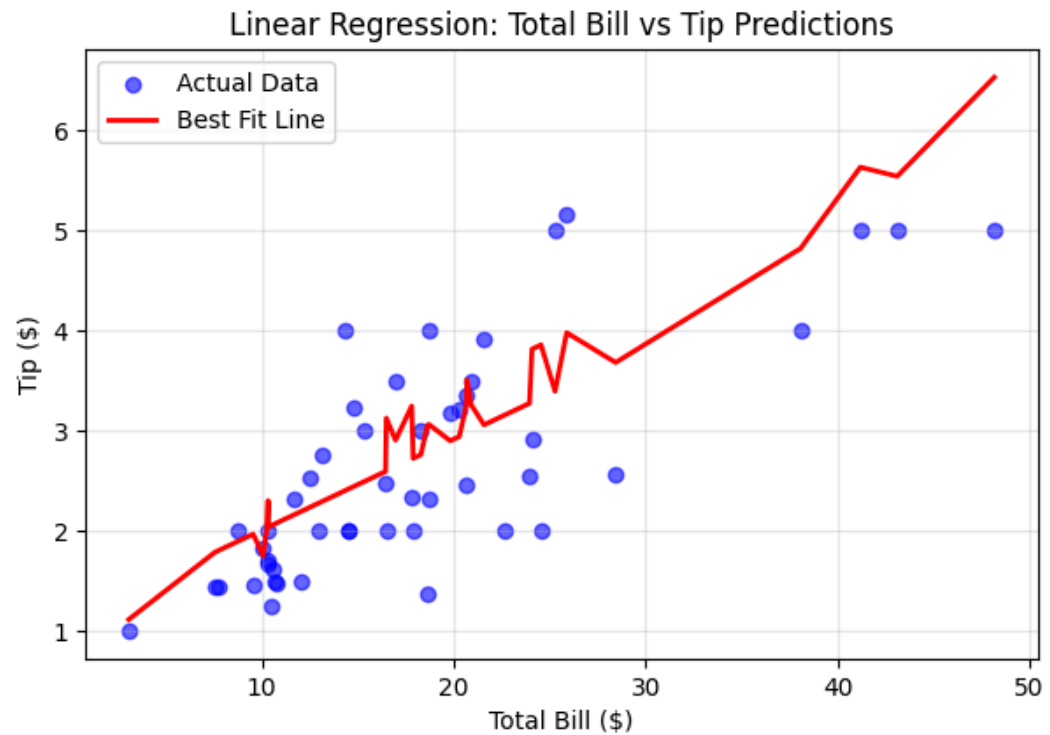
plt.title('Linear Regression: Total Bill vs Tip Predictions')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')

plt.legend()

plt.grid(True, alpha=0.3)

plt.show()
```

Output:



How to Interpret the Regression Output Above:

1. Mean Squared Error (0.65):

Mean Squared Error (0.65): This value represents the average squared difference between the actual tip and the model's prediction. If we take the square root $\sqrt{0.65} \approx 0.80$ it means the model's predictions are off by about \$0.80 (80 cents) on average from the actual tip value.

2. R-Squared Score (0.48):

If the decimal value $0.48 = 48\%$ is converted into percentage form, it becomes 48%.

- **Meaning:** The linear regression model successfully explains **48% of the variation in tip values** based on the features `total_bill` and `size`.
- **The remaining: 52%** of the variation is influenced by external factors not included in the dataset (for example: service quality, waiter friendliness, or customer mood).

4. Classification: Logistic Regression and Performance Metrics

The core goal of classification is to find a Decision Boundary. This is a boundary (it could be a line or a curve) that separates different classes in your data. For example, a boundary that separates "Healthy" patients from "Sick" patients based on their medical features.

4.1 Logistic Regression

A. The Sigmoid Function

Instead of fitting a straight line that goes to infinity, Logistic Regression uses the Sigmoid Function (also known as the S-curve). It maps any real-valued number into a value between 0 and 1.

- If the output is ≥ 0.5 , the model classifies it as Class 1 (e.g., Positive).
- If the output is < 0.5 , the model classifies it as Class 0 (e.g., Negative).

Example:

```
import numpy as np
import matplotlib.pyplot as plt

x_sigmoid = np.linspace(-10, 10, 100)

y_sigmoid = 1 / (1 + np.exp(-x_sigmoid))

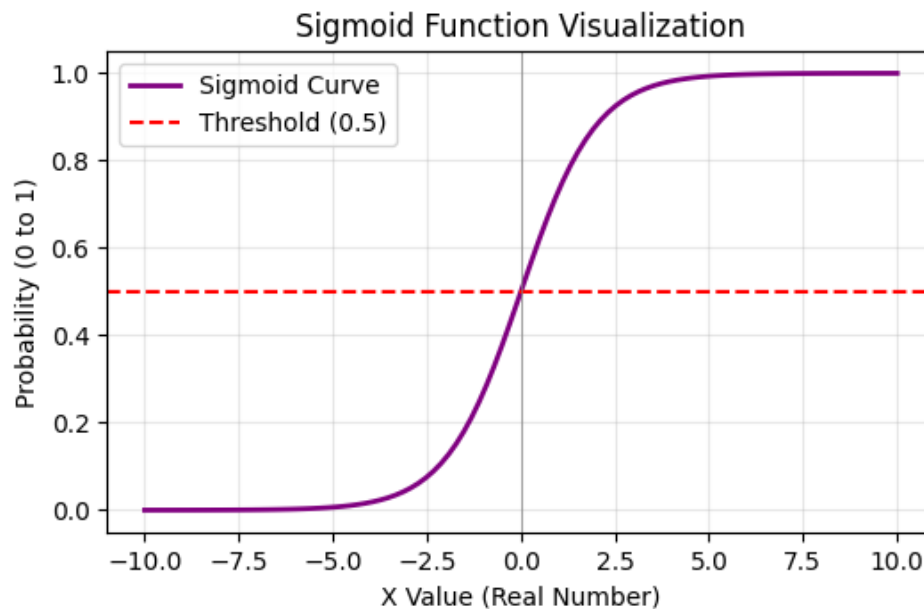
plt.figure(figsize=(6, 3.5))
```



```
plt.plot(  
    x_sigmoid,  
    y_sigmoid,  
    label='Sigmoid Curve',  
    color='purple',  
    linewidth=2  
)  
  
plt.axhline(  
    0.5,  
    color='red',  
    linestyle='--',  
    label='Threshold (0.5)'  
)  
  
plt.axvline(0, color='gray', linewidth=0.5)  
  
plt.title('Sigmoid Function Visualization')  
plt.xlabel('X Value (Real Number)')  
plt.ylabel('Probability (0 to 1)')  
  
plt.legend()  
  
plt.grid(True, alpha=0.3)  
  
plt.show()
```



Output:



B. Use Cases

- Binary Classification: Does a customer churn or stay? (Yes/No)
- Medical Diagnosis: Does the patient have the disease? (True/False)

4.2 The Confusion Matrix

In classification, we don't use MSE or R-Squared. Instead, we use a Confusion Matrix to see exactly where the model succeeded and where it got "confused."

A Confusion Matrix breaks down the results into four categories:

- True Positive (TP): Predicted "Yes" and it was actually "Yes."
- True Negative (TN): Predicted "No" and it was actually "No."
- False Positive (FP): Predicted "Yes" but it was actually "No" (Type I Error).
- False Negative (FN): Predicted "No" but it was actually "Yes" (Type II Error).

Standard Metrics Derived from the Matrix:

- Accuracy: The percentage of total correct predictions $(TP + TN) / \text{Total}$.
- Precision: Out of everyone we predicted as "Sick," how many were actually sick?
- Recall: Out of everyone who was actually sick, how many did we successfully find?

Example:

```
import matplotlib.pyplot as plt
import seaborn as sns
```



```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
confusion_matrix, accuracy_score

model_clf = LogisticRegression()

model_clf.fit(X_train_scaled, y_train_c)

y_pred_c = model_clf.predict(X_test_scaled)

print(f"Model Accuracy Score: {accuracy_score(y_test_c, y_pred_c)
* 100:.2f}%\n")
print("Classification Report:")
print(classification_report(
    y_test_c,
    y_pred_c,
    target_names=['Class 0', 'Class 1']
))

cm = confusion_matrix(y_test_c, y_pred_c)

plt.figure(figsize=(5, 3.5))

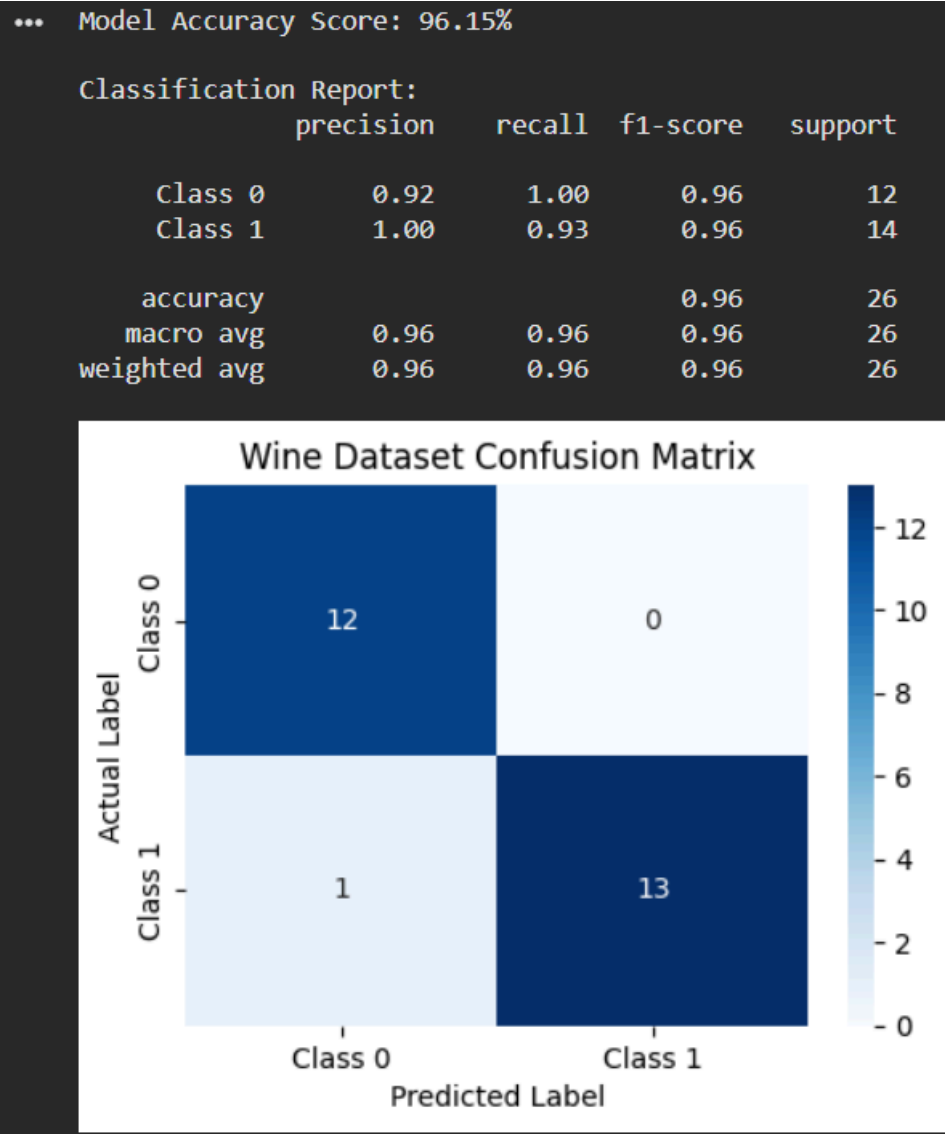
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Class 0', 'Class 1'],
    yticklabels=['Class 0', 'Class 1']
)

plt.title('Wine Dataset Confusion Matrix')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')

plt.show()
```

Output:





Precision vs Recall

- When to use Precision?
When we want to minimize false positive predictions as much as possible.
Example: Spam Classification – we do not want important emails to be classified as spam.
- When to use Recall?
When we cannot afford to miss any positive cases.
Example: Cancer Detection – it is better to mistakenly classify a healthy person as sick for further examination rather than letting a cancer patient be classified as healthy.



CODELAB (20%)

Task 1: Data Acquisition & Inspection (10 Points)

Load the Titanic dataset from the provided URL. Inspect the data to identify numerical features (X) and the target variable (y).

- Dataset URL:
<https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv>
- Target: Survived (0 = No, 1 = Yes)

```
import pandas as pd

# Load dataset from online repository
url = "----"
columns = ['---']
df = pd.read_csv("")

# Preview the first 5 rows
df.head()
```

Task 2: Data Splitting and Preprocessing (40 Points)

1. Select numerical features (Pclass, Age, SibSp, Parch, Fare) and encode the categorical feature Sex into numerical values (0 for male, 1 for female).
2. Handle missing values in the Age column by filling them with the median. Split the data into 70% training and 30% testing sets using stratify=y.
3. Apply StandardScaler to normalise the feature scales.

Task 3: Model Implementation & Metrics (40 Points)

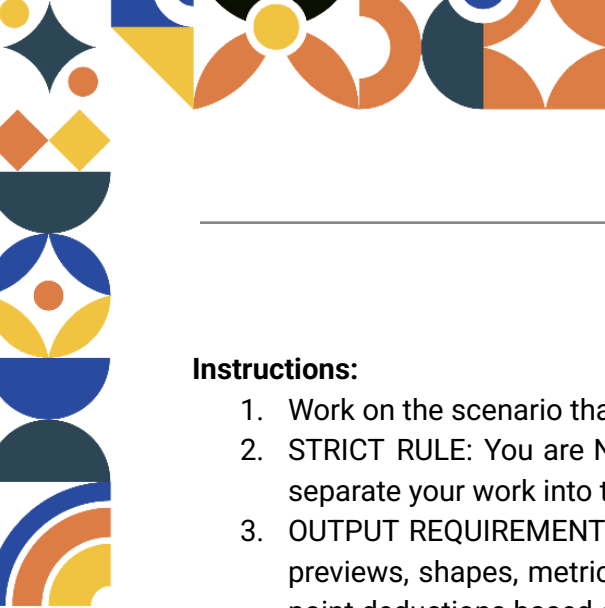
Implement a Logistic Regression model and train it on the scaled training set.

- Expectation: Calculate the Accuracy Score on the testing set. Your model is expected to achieve an accuracy score of ABOVE 70% (> 70%).
- Display the Classification Report (Precision, Recall, F1-Score) to further evaluate the performance.

Task 4: Performance Visualization (10 Points)

Create a Heatmap of the Confusion Matrix using Seaborn based on your predictions. Ensure the axes are correctly labeled ('Not Survived' and 'Survived') for clear interpretation.





LAB ASSIGNMENT (80%)

Instructions:

1. Work on the scenario that matches the last digit of your Student ID (NIM).
2. STRICT RULE: You are NOT allowed to combine your code into a single cell. You MUST separate your work into the 5 distinct cells specified below.
3. OUTPUT REQUIREMENT: Every single code cell must generate a visible output (e.g., data previews, shapes, metrics, or plots) when executed. Missing outputs will result in severe point deductions based on the Grading Criteria.

A. Scenario A (Odd NIM): Vehicle Price Prediction (Regression)

Dataset URL:

<https://raw.githubusercontent.com/amankharwal/Website-data/master/CarPrice.csv>

Tasks & Cell Structure:

1. [Cell 1 - Data Loading & Inspection]:

Load the dataset from the URL using Pandas. Display the first 5 rows to verify the data structure. (10 Points)

2. [Cell 2 - Feature Selection & Splitting]:

Select exactly three features (horsepower, curbweight, enginesize) as X and price as y . Split into an 80% training set and a 20% testing set (random_state=42). Output requirement: Print the shapes or sample counts of your training and testing sets. (20 Points)

3. [Cell 3 - Model Training & Evaluation]:

Train a LinearRegression model. Predict the test set prices and display the MSE, RMSE, and R-Squared (R^2) Score (formatted as a percentage). (50 Points)

4. [Cell 4 - Performance Visualization]:

Generate a scatter plot of the actual test data (enginesize vs price) and overlay the red Best Fit Line (Regression Line). Hint: Remember to sort the feature index for the line plot. (10 Points)

5. [Cell 5 - Markdown Analysis]:

Write a brief conclusion interpreting your R^2 percentage and what the best fit line reveals about engine size trends. (10 Points)

B. Scenario B (Even NIM): Penguin Species Classification (Classification)

Dataset URL:

<https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv>

Tasks & Cell Structure:

1. [Cell 1 - Data Loading & Filtering]:



Load the dataset from the URL. Filter the dataframe to include only 'Adelie' and 'Chinstrap' species. Drop rows with missing values (NaN). Output requirement: Display the first 5 rows of the filtered dataframe. (10 Points)

2. [Cell 2 - Feature Splitting & Scaling]:

Select four features (bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g) as X and species as y. Split 80:20 with random_state=42 and stratify=y. Apply StandardScaler. Output requirement: Print the dimensions/shapes of the scaled training and testing data. (20 Points)

3. [Cell 3 - Model Training & Evaluation]:

Train a LogisticRegression model on the scaled data. Predict the test set categories. Output requirement: Display the Model Accuracy Score and the complete Classification Report. (50 Points)

4. [Cell 4 - Performance Visualization]:

Generate and display a Heatmap of the Confusion Matrix using Seaborn with proper labels (Adelie and Chinstrap) and a title. (10 Points)

5. [Cell 5 - Markdown Analysis]:

Write a brief conclusion interpreting your confusion matrix heatmap and analyze the errors found. (10 Points)

GRADING CRITERIA & DETAILS

Activity & Details	Weight
Codelab	20%
Task 1 - Task 4	20
Lab Assignment	80%
Task 1 - Task 5	50
Explanation	30
Total	100%

